



Pour l'instant, nous avons fait un certain nombre de commits sur notre dépôt git local.

Nous avons vu comment revenir en arrière si on a fait une erreur, mais que se passe-t-il lorsqu'on a commis l'ajout de nouveaux fichiers qui ne devaient pas figurer dans le dépôt git.

Création d'un nouveau fichier

Dans le projet « first » nous allons créer un nouveau fichier appelé **mesidentifiants.log**

```
$ notepad mesidentifiants.log
```

Nous allons simplement écrire à l'intérieur des credentials

```
username: Jordan  
password: mypassword
```



Nous allons ajouter un nouveau fichier à notre projet appelé « first ». Il s'agit du fichier mesidentifiants.log qui contient mon identifiant et mon mot de passe pour accéder à mon application. C'est mal, ne faites jamais une chose pareille. Mais pour l'exemple c'est ce que nous allons effectuer.

Verification du status de notre projet git

On peut vérifier le status grâce à la commande **git status**

```
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be
  committed)

    mesidentifiants.log

nothing added to commit but untracked files present
(use "git add" to track)
```



On voit ainsi que notre fichier mesidentifiants.log est correctement détecté par git, mais qu'il ne « track » pas encore les modifications de ce fichier.

Placer le fichier mesidentifiants.log en staging et commiter

On va déplacer notre fichier mesidentifiants.log en état staging et commiter

```
$ git add mesidentifiants.log

$ git commit -m "Add file mesidentifiants.log (baaaaaad)"
[master 8d31e76] Add file mesidentifiants.log (baaaaaad)
1 file changed, 2 insertions(+)
 create mode 100644 mesidentifiants.log

$ git status
On branch master
nothing to commit, working directory clean
```



Cependant, je me rends compte que j'ai fait une erreur !!

Ce fichier n'aurait jamais dû être commité dans mon dépôt...

Supprimer un fichier dans git

On utilise la commande **git rm** pour supprimer un fichier dans git

```
$ git rm mesidentifiants.log
rm 'mesidentifiants.log'

$ ls
hello-world.py  README.md

$ git status
On branch master
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

        deleted:    mesidentifiants.log
```

On pourrait penser qu'il suffit de supprimer le fichier simplement pour l'enlever du dépôt, mais ce n'est pas aussi simple.

En effet, il faut le retirer spécifiquement de la zone du Dépôt Git, et non uniquement de la zone de travail.

Pour ce faire on utilise alors la commande **git rm** (pour remove).

Si je fais un **ls**, je peux d'ailleurs voir que le fichier `mesidentifiants.log` a bien été supprimé. Cependant, avec la commande **git status** je peux voir que la

suppression du fichier **mesidentifiants.log** doit être
commitée.

Committer la suppression du fichier

On utilise la commande **git commit** pour valider de manière permanente la suppression du fichier `mesidentifiants.log`

```
$ git commit -m "Remove mesidentifiants.log"
[master af07efe] Remove mesidentifiants.log
1 file changed, 2 deletions(-)
delete mode 100644 mesidentifiants.log
```



En réalité on peut également simplement supprimer directement le fichier depuis notre gestionnaire de fichier, ou en utilisant la commande linux « **rm** ».

On fera ainsi :

```
$ rm mesidentifiants.log
```

```
$ git add mesidentifiants.log
```

```
$ git commit -m "remove mesidentifiants.log file"
```